

Convolutional Neural Network $\rightarrow$ need?

The problem : ① Standard Network, take a small photo (200x200) pixels into a flatten row of 40,000 pixels. The network loses the 2D shape of image.

② we need billion of parameters for hidden layer connection, its inefficient and slow

The solution $\rightarrow$ we need network that looks at patches of the image (like eye do) rather than whole thing at once.

Convolutional Neural Network

A specialized Neural Network designed for processing data with a grid like topology.

Example :

1D Grid : Time series data (sample taken at regular interval)

2D Grid : Image data (pixels arranged in a grid)

Convolution vs Matrix Multiplication

Standard Network : use simple matrix multiplication (every input unit interact with every unit output)

CNN : Use a mathematical operation called Convolution.

Biological Inspiration

- CNNs are a prime example of Neuroscientific principles influencing deep learning.
- Emulate how brain works.

The convolution Operation

How can we understand convolution?

↳ A signal processing problem : Smoothing Noise

Example : A noisy Spaceship tracking

Scenario : Track a spaceship with laser sensor.

Problem : Sensor noisy, at single moment of reading $x(t)$ might be jitter or wrong

Solution : we average it with past signal reading. However, recent reading is more important than old one.

Mechanism : Recent : higher weight at timestamps
old : lower weight at timestamps

This process is called sliding a weight function over a data to create a new, smoother signal is convolution.

Mathematical Operations

A. Convolution for continuous

for any signal, data are in continuous as time is continuous. The convolution is defined as integral.

$$s(t) = \int x(a) w(t-a) da$$

where

$s(t)$ = output

$x(a)$ = The input

$w(a)$ = The kernel (the weight function)

t = current time

a = The variable of integration (the age of the measurement)

$w(t-a)$ = part flips weight function.

a is the age, then as a increase (getting older), we look further back in time.

(B) Discrete convolution

- Computer cannot handle continuous infinity
- So, discrete step were taken
- Hence, integral change to sum.

Mathematical equation,

$$s(t) = (x * w)(t) = \sum_{n=-\infty}^{\infty} x(a) w(t-a)$$

Note: In deep learning, we deal with finite array, not infinite sum.

Input x and weight w are zero everywhere except where we need.

The 2D convolution (Image)

The formula extend to ~~double~~

$$S(i, j) = (I * K)(i, j)$$

$$= \sum_m \sum_n I(m, n) K(i-m, j-n)$$



where,

$I \rightarrow$ input Image- (2D)

$K \rightarrow$ The kernel

$i, j \rightarrow$ coordinate of the output pixel

$m, n \rightarrow$ The variable we iterate over to slide the kernel across input.

Note: $K(i-m, j-n) \rightarrow$ this means flipped relative to input.

why?

① As we move forward in image, kernel logic move backward

② As convolution $\rightarrow$ required flipping

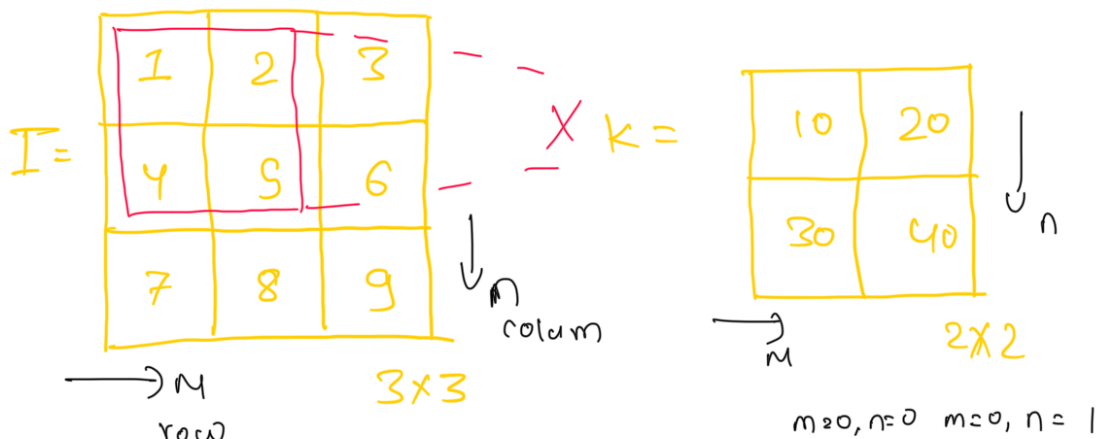
Convolution vs Cross-correlation

- Convolution actually requires flip of kernel, -commutative
- cross-correlation → does not require flip
↳ its ignore in deep learning.

So, Cross-correlation:

$$S(i, j) = (k * I)(i, j)$$

$$= \sum_m \sum_n I_{(i+m, j+n)} k(m, n)$$



position, $i=0, j=0$

$$S(i, j) = S(0, 0)$$

$$m=0, n=0 \quad = I(0,0) \times k(0,0) = 1 \times 10 = 10$$

$$m=0, n=1 \quad = I(0,1) \times k(0,1) = 2 \times 20 = 40$$

$$m=1, n=0 \quad = I(1,0) \times k(1,0) = 4 \times 30 = 120$$

$$m=1, n=1 \quad = I(1,1) \times k(1,1) = 5 \times 40 = 200$$

$$\text{Sum} = 10 + 40 + 120 + 200 = 370$$

Note 3 Convolution convert the matrix in the form of Toeplitz matrix

$$\begin{bmatrix} k_1 & k_2 & 0 & 0 \\ 0 & k_1 & k_2 & 0 \\ 0 & 0 & k_1 & 0 \\ 0 & 0 & 0 & k_2 \end{bmatrix}$$

Padding

A technique used to preserve the spatial dimensions of the input image after convolution operations on feature map.

Two way:

- ① valid : ① no padding to feature map
② feature map of out put smaller than input feature map.
③ reduce spatial dimension of the feature.

② same padding :

- ① padding added
- ② size of input feature map equal to output feature map
- ③ preserve spatial dimension

Stride

↳ How many step we should take to slide our filter?

- ① Small step detail information
- ② Bigger step reduce feature size
↳ reduce computation size

formula :

$$\frac{k-1}{2}$$

$k \rightarrow$ size of kernel

Pooling :

This is used to reduce the spatial dimension (width and height) of the input feature while retaining the most important features.

why?

- ① Dimensionality Reduction
- ② Translation Invariance
- ③ Overfitting prevention
- ④ feature hierarchy.

Types :-

- ① Max pooling
- ② Average pooling
- ③ Global pooling

$\hookrightarrow$ max Global pooling

$\hookrightarrow$ Average Global pooling

Backpropagation $\rightarrow$

Forward

$\hookrightarrow$

1 2 3 4 5 6 7 8 9 10

a_1	a_2	a_3	a_4
a_5	a_6	a_7	a_8
a_9	a_{10}	a_{11}	a_{12}
a_{13}	a_{14}	a_{15}	a_{16}

4x4

*

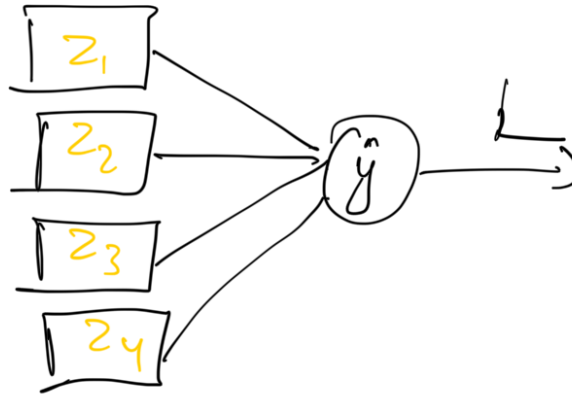
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

=

z_1	z_2
z_3	z_4

z_1	z_2
z_3	z_4

=>



Always

$$w_i^* = w_i - \alpha \times \frac{\partial L}{\partial w_i}$$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$\frac{\partial L}{\partial w_1}$	$\frac{\partial L}{\partial w_2}$	$\frac{\partial L}{\partial w_3}$
$\frac{\partial L}{\partial w_4}$	$\frac{\partial L}{\partial w_5}$	$\frac{\partial L}{\partial w_6}$
$\frac{\partial L}{\partial w_7}$	$\frac{\partial L}{\partial w_8}$	$\frac{\partial L}{\partial w_9}$

$\frac{\partial L}{\partial z_1}$	$\frac{\partial L}{\partial z_2}$
$\frac{\partial L}{\partial z_3}$	$\frac{\partial L}{\partial z_4}$

$$\frac{\partial L}{\partial w_1} = \underbrace{\frac{\partial z_1}{\partial w_1}}_{a_1} \times \frac{\partial L}{\partial z_1} + \underbrace{\frac{\partial z_2}{\partial w_1}}_{a_3} \times \frac{\partial L}{\partial z_2} + \underbrace{\frac{\partial z_3}{\partial w_1}}_{a_{11}} \times \frac{\partial L}{\partial z_3} + \underbrace{\frac{\partial z_4}{\partial w_1}}_{a_{13}} \times \frac{\partial L}{\partial z_4}$$

$\partial 1$

$$\frac{\partial L}{\partial w_2}$$

$$\frac{\partial L}{\partial w_3}$$

$$\frac{\partial L}{\partial w_4}$$

⋮

$$\frac{\partial L}{\partial w_9}$$

1

where

$$Z_1 = w_1 \times a_1 + w_2 \times a_2 + \dots + w_9 \times a_9$$

⋮

$$Z_4 =$$

w_1^*	w_2^*	w_3^*
w_4^*	w_5^*	w_6^*
w_7^*	w_8^*	w_9^*

=

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$-\alpha \times$

$\frac{\partial L}{\partial w_1}$	$\frac{\partial L}{\partial w_2}$	$\frac{\partial L}{\partial w_3}$
$\frac{\partial L}{\partial w_4}$	$\frac{\partial L}{\partial w_5}$	$\frac{\partial L}{\partial w_6}$
$\frac{\partial L}{\partial w_7}$	$\frac{\partial L}{\partial w_8}$	$\frac{\partial L}{\partial w_9}$

How can you design CNN in pytorch

Input image: $1 \times 32 \times 32$

Class CNN_layer(nn.Module):

def __init__(self):

```
super(conv_layer, self).__init__()
```

```
# Layer 1, in out kernel stride
```

```
self.conv1 = nn.Conv2d(1, 6, 5, 1)
```

$$\left[\frac{32-5}{1} + 1 \right] = 28$$

output shape = (6, 28, 28)

```
# Avg pool = layer 2
```

```
self.pool1 = nn.AvgPool2d(kernel_size=2,  
                           strides=2)
```

$$\left[\frac{28-2}{2} + 1 \right] = 14$$

(6, 14, 14)

```
# conv2d - layer 3
```

```
self.conv2 = nn.Conv2d(6, 16, 5, 1)
```

$$\left[\frac{14-5}{1} + 1 \right] = 10$$

output (16, 10, 10)

```
# Layer 4
```

```
self.pool2 = nn.AvgPool2d(kernel_size=2,  
                           stride=2)
```

$$\left[\frac{10-2}{2} + 1 \right] = 5$$

output $\Rightarrow$ (16, 5, 5)

```
# fc 1  $\rightarrow$  linear
```

```
self.fc1 = nn.Linear(16 * 5 * 5, 120)
```

```
# fc 2  $\Rightarrow$  linear,
```

```
self.fc2 = nn.Linear(120, 84)
```

Output

self.fc3 = nn.linear(84, ^{classes,} 10)

def forward(self, x):

x = self.pool1(torch.relu(self.conv1(x)))

x = self.pool2(torch.relu(self.conv2(x)))

x = x.view(-1, 16 * 5 * 5) # flatten

x = torch.relu(self.fc1(x))

x = torch.relu(self.fc2(x))

x = self.fc3(x)

return x